Full-Time-Pad Symmetric Stream Cipher

Built on One-Time-Pad Encryption Scheme

Taha Canturk
Department of Electrical and Computer Engineering
University of Waterloo
tcanturk@uwaterloo.ca

Abstract

Essential properties in ciphers include computational efficiency, security, and economical feasibility. Encryption scheme based on one-time-pad (OTP) is secure, but there are two security risks: (1) a key cannot be reused and (2) plaintext length equals the key length which is costly & inefficient for long plaintexts. As stated by Claude Shannon, father of information theory [1], a secure cryptographic algorithm requires both confusion and diffusion. However OTP lacks the confusion and diffusion per ciphertext. In this study, we proposed the **full-time-pad (FTP)**, a symmetric stream cipher that extends OTP by incorporating both diffusion and confusion. To achieve diffusion, symmetric key is permuted in its byte form using a constant permutation matrix. The permutation ensures that every 32-bit number is derived from different bytes of the key. Using modern hardware optimizations (Vectorization, Cache and Parallelization, this operation is computationally efficient. To achieve confusion, the key is manipulated in 32-bit segments using Modular **A**ddition, Bitwise **R**otations, and **X**or (**ARX**).

Contents

1 Introduction **1**
- 1.1 Key Generation . . . 1
- 1.2 Prerequisite Mathematics . . . 1
  - 1.2.1 Addition in the ring $\mathbb{Z}_{2^{32}}$ . . . 1
  - 1.2.2 Vector Permutation . . . 1

2 Security **2**
- 2.1 NIST SP 800-22 - Statistical Test Suite - Official NIST Randomness Test . . . 3
- 2.2 Brute-Force . . . 3
  - 2.2.1 Birthday Problem . . . 3
  - 2.2.2 Collision Attack . . . 6
- 2.3 Joux's Attack: Multicollisions in Iterated Hash Functions . . . 8
- 2.4 Related Key Attack . . . 9
- 2.5 Time Based Attacks . . . 9
- 2.6 Linear Cryptoanalysis . . . 9
- 2.7 Differential Cryptoanalysis . . . 10
- 2.8 Reverse Engineering the Transformation . . . 13
- 2.9 Side-Channel Attacks . . . 13
- 2.10 Collision-Resistance . . . 13
  - 2.10.1 Different Permutation Matrices . . . 14
  - 2.10.2 Number of Rounds . . . 14
  - 2.10.3 Constant - r - Dynamic Rotation Constant . . . 14

3 Hashing **15**
- 3.1 Dynamic Matrix Permutation . . . 15
  - 3.1.1 Derivation . . . 16
  - 3.1.2 Dynamic Permutation Matrix Values . . . 16
- 3.2 Confusion - ARX . . . 17
  - 3.2.1 A - Modular Addition . . . 17
  - 3.2.2 R - Bitwise Rotation . . . 17
  - 3.2.3 X - XOR . . . 17

4 Performance **17**

5 Cipher **19**
- 5.1 Transformation - Version 1.0 - Prioritize Complexity . . . 19
- 5.2 Transformation - Version 1.1 - Prioritize Speed & Complexity . . . 19
- 5.3 Transformation - Version 2.0 - Prioritize Speed . . . 20
- 5.4 dynamic permutation . . . 21
- 5.5 endian_8_to_32_arr . . . 22
- 5.6 Key Reuse . . . 22
  - 5.6.1 Multi-Threading . . . 22
- 5.7 Long Plaintexts . . . 23
- 5.8 Encryption/Decryption operation . . . 23

6 Conclusion **23**

A Pre-requisite Terminology **24**

1 Introduction

Fast & efficient encryption algorithms are a desired concept for both large and small amounts of data. Single Instruction Multiple Data (SIMD) instructions in modern devices are a golden opportunity for hardware optimized cryptographic algorithms. Even though newer algorithms like Blake-3 utilize it, their benefits aren't maximized. According to benchmark tests on FTP, most cache misses happen in the first round of permutation, next permutations are consistently faster which is why using multiple permutations are a worthwhile approach. FTP is three times faster than ChaCha20 because it is built for modern hardware. SIMD Parallelization has been around since 1962 [2], yet they have been popularized since the early 1990s [6]. SIMD exists in nearly all modern devices and we present that cryptography should adapt to this modernization. One of the main reasons why multiple rounds of permutations aren't considered in previous works is because diffusion relied on transposing bytes located in RAM while FTP relies on cache and SIMD optimizations. In the Efficient Permutation Instructions for Fast Software Cryptography [3] publication from Princeton University, bit-level permutation optimization are introduced. Such algorithms would significantly reduce the number of rounds needed for a cryptographic algorithm such as ChaCha20 which includes 20 rounds of transformations yet FTP only requires 10 rounds. This is because of the byte-level permutation mixed with simple ARX operations. However, hardware bit-level permutation optimizations aren't offered in personal devices as of now, and should only be considered for future algorithms. The basic terminology used in this study is given in Appendix A.

1.1 Key Generation

The 32-byte key should be generated using a cryptographically secure method, including but not limited to cryptographic random number generators and Hash-based Key Derivation Functions (HKDF) depending on the context.

1.2 Prerequisite Mathematics

1.2.1 Addition in the ring $\mathbb{Z}_{2^{32}}$

In an integer ring, where two variables (x, y) are added, $x, y \in \mathbb{Z}_{2^{32}}$ where $\mathbb{Z}_{2^{32}}$ denotes the ring with size 2^{32}.

The expression:
$x + y = z$
$x, y, z \in \mathbb{Z}_{2^{32}}$

Implicitly denotes
$x + y = z \pmod{2^{32}}$

So all addition operations require modulo 2^{32}.
In the context of this encryption algorithm, it's used in the transformation algorithm as one of four operations (**A**RX).

1.2.2 Vector Permutation

Permuting a vector refers to transposing the elements in a vector. It is a reversible operation. For example, let V be the vector to transpose:

$$V = (0\ 1\ 2\ 3)$$

let P be the permutation vector:

$$P = (3\ 2\ 1\ 0)$$

then the resulting permuted vector would be

$$V' = (3\ 2\ 1\ 0)$$

Observe V_0 ($\mathbf{0}$), according to P_0, it should be replaced with the 3rd value in P. so then $V'_0 = 3$. This doesn't mean that $V'_3 = 0$. This algorithm is demonstrated in dynamic permutation section (5.4). The permutation in this algorithm is used for the bytearray `key` which offers diffusion for the 32-bit segmented `k`. In order to confuse and diffuse, `k` and `key` must point to the same memory address. Most efficient method to accomplish this is to use `reinterpret_cast` in C++ or `unions` in C. Since these are low level approaches, endianness of the system will affect the byte order. In a little-endian system, the data requires the following permutation before the transformation iterations:

$$V' = (3\ 2\ 1\ 0\ 7\ 6\ 5\ 4\ 11\ 10\ 9\ 8\ 15\ 14\ 13\ 12\ 19\ 18\ 17\ 16\ 23\ 22\ 21\ 20\ 27\ 26\ 25\ 24\ 31\ 30\ 29\ 28)$$

2 Security

In OTP, key cannot be reused. This is demonstrated by the following:

`let` m_1, m_2 `be 2 plaintexts`
`let` k `be the key`
`let` $c_1 = m_1 \oplus k$
`let` $c_2 = m_2 \oplus k$
$c_1 \oplus c_2 = (m_1 \oplus k) \oplus (m_2 \oplus k)$
$c_1 \oplus c_2 = m_1 \oplus m_2$

Since the key is reused, the 2 ciphertext's XORed factor out the key since $k \oplus k = 0$ Using cryptoanalysis, the 2 plaintexts can be found as demonstrated by OneTimePad Cracker on GitHub by Kibnakamoto.
To imply that $c_1 \oplus c_2 \neq m_1 \oplus m_2$ implicitly requires that the key is updated per encryption. If k is transformed each time so that it has an avalanche effect. Even with no confusion, it would still be secure since $k' \oplus k \neq 0$ where k' is transformed key.
But there is another concern,
If the plaintext and ciphertext are known (known plaintext attack), then it is possible to find k so avoid using k without transformation since `plaintext` $\oplus$ `ciphertext` = `key`, for each plaintext,

key needs to be transformed irreversibly and also requires confusion since if k' is found, k is still unknown but if k is found, then all instances of k'_n are known, which means that:

$k'_1 = \text{hash}(k+1)$ `where hash() is an irreversible transformation`
$k'_2 = \text{hash}(k+2)$
$c_1 \oplus c_2 = (m_1 \oplus k'_1) \oplus (m_2 \oplus k'_2)$
$c_1 \oplus c_2 \neq m_1 \oplus m_2$
$m_1 \oplus c_1 = k'_1$
$m_2 \oplus c_2 = k'_2$
k'_1, k'_2 `are calculated using an irreversible hashing algorithm`

$\therefore$ `FTP` requires both diffusion and confusion

2.1 NIST SP 800-22 - Statistical Test Suite - Official NIST Randomness Test

Using the python tool from https://github.com/stevenang/randomness_testsuite, all 3 versions of the transformation algorithm have been tested using a patterned key ($\texttt{key}_i = i, i = 0, 1, 2, \ldots, 31$). Tests were conducted for both small changes in key and encryption index. For each 256-bit segment until the one millionth bit, the input was incremented once per encryption. All 3 versions of the transformation function resulted in full-randomness for all of the 16-tests. This proves that the slightest change in input (key/encryption index) demonstrates a strong randomness, and therefore has a proper avalanche effect.

2.2 Brute-Force

2.2.1 Birthday Problem

The birthday problem is a well known veridical paradox. It goes as follows: how many people are required so that there is a 50% chance that at least 2 people have the same birthday.
The answer is 23 people.
In the context of this encryption algorithm, this suggests that brute-forcing using random inputs is unintuitively easy which implies the need for a mathematical proof:

`let` k `be the number of keys needed for hash(key) to have a 50% chance to equal another hash(key)`
`let` b `be the number of combinations per key`
`let` V_c `be the total number of combinations for k keys with order and no repetitions`
`let` V_t `be the total number of combinations for k keys with order and with repetitions`

$$V_c = \frac{b!}{(b-k)!}$$

$$V_t = b^k = 2^{256^k}$$

let P(A) be the probability of not shared birthdays.

$$P(A) = \frac{V_c}{V_t}$$

Let $P(B)$ denote the probability of shared birthdays.

$$1 - P(B) = P(A)$$

For $P(B) = 50\%$,

$$\frac{1}{2} = \frac{b!}{b^k(b-k)!}$$

$$\log_b \frac{1}{2} b^k = \log_b \frac{b!}{(b-k)!}$$

$$\log_b \frac{1}{2} + \log_b b^k = \log_b b! - \log_b (b-k)!$$

$$k + \log_b k + \log_b (b-k)! = \log_b b! - \log_b \frac{1}{2}$$

According to Ramanujan's Approximation:

$$\log_x x! \approx \frac{x \ln x - x + \frac{\ln\left[\frac{1}{\pi^3} + x(1+4x(1+2x))\right]}{6} + \frac{ln\pi}{2}}{\ln x}$$

We can simplify the formula to the following:

$$\text{let } C = b \ln b + \frac{\ln\left[\frac{1}{\pi^3} + b(1+4b(1+2b))\right]}{6} - \ln b \log_b \frac{1}{2} \approx (b-k)\ln(b-k) + k + \ln bk + \frac{\ln\left[\frac{1}{\pi^3} + (b-k)(1+4(b-k)(1+2(b-k)))\right]}{6}$$

$$\text{let } g(b,k) = (b-k)\ln(b-k) + k + \ln bk + \frac{\ln\left[\frac{1}{\pi^3} + (b-k)(1+4(b-k)(1+2(b-k)))\right]}{6} - C(b) = 0$$

To verify the accuracy of this equation, it can be compared to a known approximation solution [5].

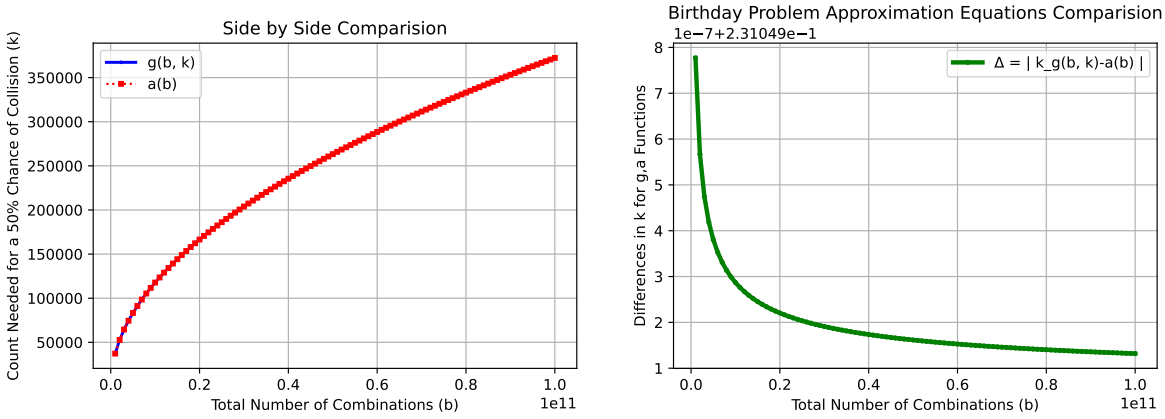
$$a(b) = \frac{1}{2} + \sqrt{\frac{1}{4} + 2\ln 2b}$$

And an accurate independent verification solution $f_\Gamma(b,k)$ to solve the f_B function defined above with Γ functions since $b, k \notin \mathbb{Z}$ for exact solution.
To compare functions a and g:

1. Provide both functions with inputs for parameter b that increments by an arbitrary integer (n) for a 100 iterations.
2. Graph their values and the differences in k.
3. Find k using another approved method. E.g. solving the birthday problem with f_Γ.

(**1.-2.**): Increment b by n=1,000,000,000 for 100 iterations and graph (b, k) for g, a:

Figure 1: Difference in functions g,a V.S Total Combination



Graph Δ shows that the difference in k for g, a get smaller as $b \to \infty$. Ramanujan approximation is the most accurate solution for factorials, so g is the most accurate solution to the birthday problem. All together, this suggests that g is more accurate for smaller values for b and that accuracy for a approaches the accuracy of g as $b \to \infty$.

(**3.**): The accuracy tests for g,a measured with f_Γ:
For each iteration (i), first check if $k_{g(b_i,k)}$ and $a(b_i)$ result in the same floor(k) since k is technically a discrete integer and if they're not equal, whichever function provides result closer to 50 % for the given expression is the most accurate function: $|f_\Gamma(b_i, k_r)|$ where k_r is the resultant k and is equal to either $k_{g(b_i,k)}$ or $a(b)$ and $b_i = i \times n$. The tests results show that g is more accurate around 16% (varying between 10-30%) of the time as $\Delta \to 0$, and $b \to \infty$. The small flaw with this is that since function g is the most accurate solution, even $f_\Gamma(b, k)$ function isn't going to show the exact accuracy. So the accuracy of g can only be determined accurately relative to a obviously less accurate function like function a.
The results:

```
g is more accurate:  True
g is closer/equal to zero than a:  100
a is closer/equal to zero than g:  84
```

The accuracy of a is off mainly because $\Delta \to \frac{\ln 2}{3}$ as $b \to \infty$. This is a strange phenomenon that is proven to be not related to function g since $f_\Gamma(b, k) \approx g(b, k) \approx a(b) + \frac{\ln 2}{3}$. a is always off by the same value because of how that a is derived. This means that a can be accurate 100% of the time as long as $a(b) + \frac{\ln 2}{3}$ is used rather than just $a(b)$ so that a returns k that's only off by a negligible amount for most contexts.

The code for this test is in **test/sec.py**.

$\therefore g(b, k)$ `can be used to evaluate how many keys it would take so that 2 hashes have a 50% chance of being equal.` $g(b, k)$ `can be evaluated using the secant algorithm (test/secant.py)`

After running `test/secant.py`, given the parameters:
Based on approximations in the Wikipedia Article: Birthday Problem, approximate x_0 and x_1
$x_0 = \frac{1}{2} + \sqrt{\frac{1}{4} + 2 \times ln(2) \times b}$ (due to Approximation of number of people)
$x_1 = \sqrt{b}$ (due to square approximation)

error tolarance: $e = 1 \times 10^{-200}$

Assuming a computation speed of 1 trillion keys per second, it would still take 10^{27} seconds ($\frac{k_1}{1 \text{ trillion} \frac{\text{keys}}{\text{s}}}$) to try all keys so that there is a 50% chance of collision. Therefore brute-forcing the key is not possible.

2.2.2 Collision Attack

Most Denial of Service (DOS) attacks related to encryption algorithms are based on brute-force methods. Following is the proof that there are no potential collision attack vulnerabilities in the form:
`transform(`key_1`)` = `transform(`key_2`)`

Addition can be observed as an encryption algorithm. For example: $x + y = 16$

$x, y \in \mathbb{Z}, 0 \leq x, y < 256$
there are 17-combinations for x to satisfy this expression, and simultaniously, there are 17 combinations for y to satisfy the expression, so there is a total of 17 combinations.
But for $x + y = z$, there are 256 combinations to try. if the result of an arithmetic operation is known, there may be ways to get the same end-result with less combinations to brute-force. Knowing the value of z reduced the number of combinations by 15 times.

$\therefore$ if there is an operation that can provide the same output for a wide range of inputs, there can be a collision attack. Collision attacks can be used to derive the same transformed key using a different input key and decrypt the plaintext without actually having the original key.
In the context of this encryption algorithm (using addition as an example operation): keysize is 32-bytes so for each byte n: $x_n + y_n = z_n$
$x, y, z \in \mathbb{Z}, 0 \leq x, y, z < 256$

Number of combinations can be represented by

$$\prod_{n=0}^{32-1} (z_n + 1)$$

so the number of combinations would be between a minimum of 32 combinations ($z_n = 0$ for all 32-bytes) up to a maximum of 2^{256} combinations ($z_n = 255$ for all 32-bytes) which can be brute forced for small z_n. So a simple addition is prone to collision attacks for $x_n + y_n = z_n$, where x, y are unknown.

$$\sum_{i=0}^{z_n} x_i + y_i \equiv z_n \Longrightarrow \sum_{i=0}^{z_n} (z_n - i) + (i) = z_n$$

Solves for all possible x, y values for each z_n. An addition operation can be represented differently to solve for 2 unknowns, while a good mix of ARX operations cannot be reverse engineered. Addition of encryption index + key might lead to vulnerabilities. Also pre-manipulation of key to other values such as a constant would be infeasible because it would provide no further security benefits.

Therefore the alternative final secure solution is to calculate sum of each 32-bit segment of the key (represented by k_i) in order to interlink them to make sure that every byte of the key offers the same avalanche effect:

$$\sum_{i=0}^{7} k_i$$

To stress test if this concept offers enough collision resistance: think of this problem as an example:
1. $x + y = 16$ `offers n=17 combinations`
2. $x + y + z = 16$ `offers n=153 combinations (determined experimentally)`
3. $x + y + z + v = 16$ `offers n=969 combinations (determined experimentally)`

$x, y, z, v \in \mathbb{Z}, 0 \leq x, y, z, v < 256$

So there has to be an equation or algorithm to summarize the relationship between number of variables (ℓ) and the sum of the addition operation (16);

Knowing that equation **1.** is the simplest equation and it offers 17 combinations. Then if the rest of the equations are represented in 2-variable fashion. number of combinations n can be derived:

For equation **2.**: There are 3 ways to represent as 2-variable equation

$$x + y \qquad x + z \qquad y + z$$

For equation **3.**: There are 6 ways to represent as 2-variable equation

$$x + y \qquad x + z \qquad x + v$$

$$y + z \qquad y + v \qquad z + v$$

The number of ways a multi-variable equation can be represented as a 2 variable equation can be summarized by the following:

$$\sum_{i=1}^{\ell-1} i$$

There is a direct corrolation between the number of combinations and the ratio between the current number of ways to represent as 2-variable equation over the previous number of ways to represent as a 2-variable equation:

$$n_\ell \approx\propto \frac{\sum_{i=1}^{\ell-1}}{\sum_{i=1}^{\ell-2}} \times k \qquad \texttt{where k is a multiplier:} k \in \mathbb{N}$$

Using more number crunching: We found the following recursive formula that finds the number of combinations that satisfies x+y+... = 16:

$$n_\ell = (n_{\ell-1} \frac{\sum_{i=1}^{x}}{\sum_{i=1}^{x-1}} + 17) \times 3 - 17 \times 3((x+1) \mod 2)$$

where $x = \ell - 1$ and x should be incremented until correct answer is reached for $\ell \geq 6$ and $n_{\ell-1}$ is previous number of combinations. This formula doesn't translate to cases where the 2-variable equation doesn't have 17 combinations.

Simply put this equation couldn't be used accurately, it can only be an approximation. But upon further number crunching, the following formula was derived that satisfies all cases:

$$\prod_{i=0}^{\ell-1} \frac{(x+i)}{1+i}$$

where x is the number of combinations for 2 variable equations. e.g. for $a+b=16, x=16+1=17$.

Which leads to the following equation. The equation contains a costly operation (factorials) so it is computationally slower, but for approximation of really large numbers (e.g. sum of billions of variables), this is preferred as it is hard/unnecessary to calculate the exact answer using the formula when ℓ or c is unrealistically large:

$$\frac{(\ell+c)!}{c!\ell!}$$

Using the Ramanujan Approximation:

$$n! \approx \sqrt{\pi}\left(\frac{n}{e}\right)^n \sqrt[6]{8n^3+4n^2+n+\frac{1}{30}}$$

The following approximation equation can be derived for really large calculations:

$$\begin{aligned}
&\text{let } t = \frac{(\ell+c)!}{c!\ell!} \\
&t \approx \frac{\sqrt{\pi}\left(\frac{c+\ell}{e}\right)^{c+\ell} \sqrt[6]{8(c+\ell)^3+4(c+\ell)^2+(c+\ell)+\frac{1}{30}}}{\sqrt{\pi}(\frac{c}{e})^c \sqrt[6]{8c^3+4c^2+c+\frac{1}{30}})\sqrt{\pi}(\frac{\ell}{e})^\ell \sqrt[6]{8\ell^3+4\ell^2+\ell+\frac{1}{30}})} \\
&t = \frac{(c+\ell)^{c+\ell}}{\sqrt{\pi}c^c\ell^\ell} \sqrt[6]{\frac{8(c+\ell)^3+4(c+\ell)^2+(c+\ell)+\frac{1}{30}}{\left(8c^3+4c^2+c+\frac{1}{30}\right)\left(8\ell^3+4\ell^2+\ell+\frac{1}{30}\right)}}
\end{aligned}$$

The equation t defined above can be used to find the approximate number of combinations in an addition expression with ℓ number of variables with a sum of c.

Using the formula for the context of this encryption algorithm:
maximum 32-bit number:

$$c = 2^{32} - 1 = \texttt{0xffffffff}$$

where $\ell = 8$ since 8 32-bit segments to the 256-bit key then, $x = c + 1$

the total number of combinations according to the equation is between 1 and
$28718276287746698572837990721805747179039464327937453310303457477163745 28 \approx 2.9 \times 10^{72}$

Which isn't possible to brute force provided key is random and not chosen to be a small value.

2.3 Joux's Attack: Multicollisions in Iterated Hash Functions

This attack requires an existence of a collision such that `transform(`k_1`) = transform(`k_2`)`. For any secure one way cryptographic function, the only way to successfully exploit the potential vulnerability is brute force. Therefore, the Joux's attack [4] isn't possible.

As proven in sections 2.2.1 and 2.2.2, this is infeasible as k_1 and k_2 cannot be evaluated. Also experimentally proven in `test/collision.cpp`. The values in `test/collision_report` show that over lots of iterations, getting the same hash is still impossible.

2.4 Related Key Attack

Related key attacks require that `transform(`k_1`)` must be similiar to `transform(`k_2`)` where k_1 and k_2 have a known difference. Evaluation of such possibilities have been disproven in sections 2.6 and 2.7

2.5 Time Based Attacks

Time based vulnerabilities are exploited by the deduction of numbers present in the algorithm from computation benchmarks. Time based attacks are used to find a side-channel vulnerability (hardware/software) where the attacker approximates the value of the key by timing the calculation speed.
Arithmetic operations such as multiplication contains vulnerabilities since multiplication not a constant time operation. But arithmetic such as ARX operations are always constant time which is why the selected confusion/diffusion operations are chosen.

Experimentally, possibility of time-based vulnerabilities can be evaluated using the following algorithm: Start from 32 zeros for a key, then incremented each byte by 1 until there are 32 255s and test again.

```
-----TESTING SIDE CHANNEL ATTACKS-----
TESTING TRANSFORMATION VERSION 1.0:  PASSED (NO SIDE CHANNEL): data is consistent
TESTING TRANSFORMATION VERSION 1.1:  PASSED (NO SIDE CHANNEL): data is consistent
TESTING TRANSFORMATION VERSION 2.0:  PASSED (NO SIDE CHANNEL): data is consistent
```

This algorithm is implemented in `test/benchmark.cpp` in the function `benchmark_hash_time_attack_v()`. This test passed, which was never in doubt. The keys with small numbers have the same speed as keys with large numbers, therefore algorithm is proven to be constant time.

Tested for time-based side-channel attacks by comparing the computation time for different key values, If computation time for smaller key (32 0s) approximately equals to computation time for (32 255s) key, then data is consistent.

2.6 Linear Cryptoanalysis

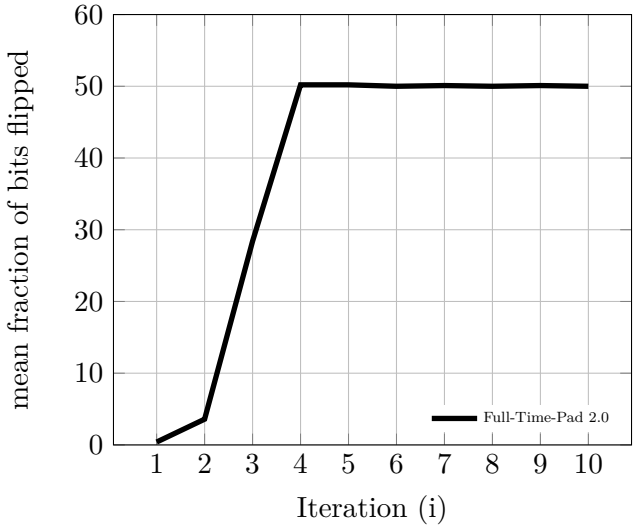
Linear Cryptoanalysis is used to find linear connections between ciphertexts and plaintexts. Linear differences in algorithms are easily exploitable weakness. Linear operations in this algorithm are byte-permutations, bitwise rotations, and xor which overall bring a strong diffusion. But not sufficient for confusion. So using addition in the ring $\mathbb{Z}_{2^{32}}$ is necessary to pass linearity tests due to carries. It's important to note that prime galois field addition is considered linear under $\mathbb{F}_{|}$ and do not bring non-linearity.
Initially, using linear algebra definition of a linear transformation, this algorithm is proven to be non-linear since `transform`$(c_1k_1 + c_2k_2) \neq c_1$`transform`$(k_1) + c_2$`transform`$(k_2)$. This doesn't mean that they lack correlation with probabilities, so further statistical analysis is required.

In order to find linear patterns with statistical analysis, using well-known proven algorithms is a solid approach.

One linear pattern is how many outputs bits flip per input bit flip. This test is known as the Strict Avalanche Criterion (SAC). SAC test passed with a 50% chance of bit flip for every output bit for each input bit.

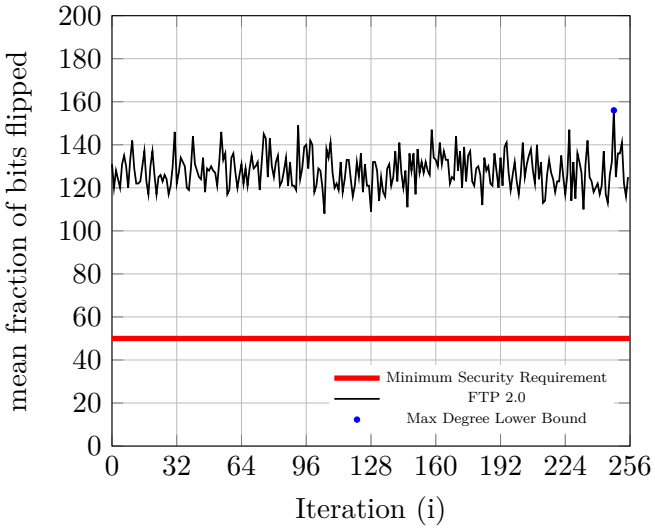
SAC can be further analyzed by checking independence of output bit flips using the Bit Independence Criterion (BIC) test. This test passed successfully and the xor of each 2 output bits follows a non-linear random pattern.

Figure 2: Bit flip probabilities vs. Iteration



To determine the convergence of bit flip probabilities per iteration, observation of the round by round SAC test is sufficient. The results of test per iteration is shown in figure 2.

Figure 3: Bit flip probabilities vs. Iteration



Final conducted linearity test is the algebraic degree which measures the algebraic degree lower bound per output bit: The tests were conducted for both changes in encryption index and key and they both provide approximetely same numbers. The same test was also conducted on ChaCha20 for reference which resulted in approximately the same output.

Although the linearity tests conducted are industry approved, they don't guarantee linearity and therefore further analysis is required to prove non-linearity.

2.7 Differential Cryptoanalysis

Differential cryptoanalysis is for measuring how input changes the output. The expected pattern should be random and should match results of secure algorithms

Given that running `test/significant_perm_byte -r` proves experimentally that all bytes of the key provide approximately the same collision resistance and that there is no pattern. This means that modifying an arbitrarily selected byte isn't only affecting some bytes but all of them without a pattern, therefore there are no significant bytes. And all bytes of the key affect the result equally. This is mainly due to the use of sum of k_i in the transformation algorithm.

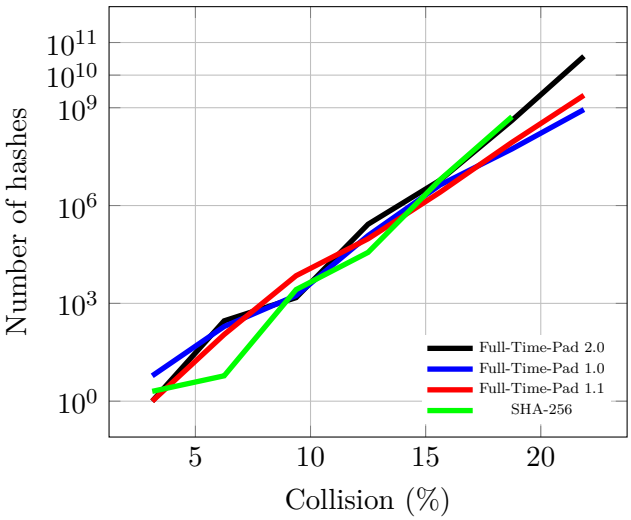
Comparative statistical data on collisions show that similiar keys (1-bit difference) contain some equivalent bytes. If all bytes of the transformed keys are equal, there is a successful collision. i is the number of hashes that are generated to get n% equality.

%	**Full-Time-Pad (i)**	**SHA-256 (i)**
3.125	1	2
6.25	291	6
9.375	1501	2648
12.5	267184	36595
15.625	5981439	6167198
18.75	408355721	530210645

Table 1: collision (%) VS. number of hashes

Random noise makes the number of hashes fluctuate a lot and relative to SHA-256, probabilities of collisions are close which suggests that the avalanche effect is real and that the math is valid for the birthday problem, therefore it's not possible to find a collision.

Figure 4: Collision (%) VS. Number of Hashes for Full-Time-Pad



Visualization of the data shows the trend of #hashes per increase in collision.

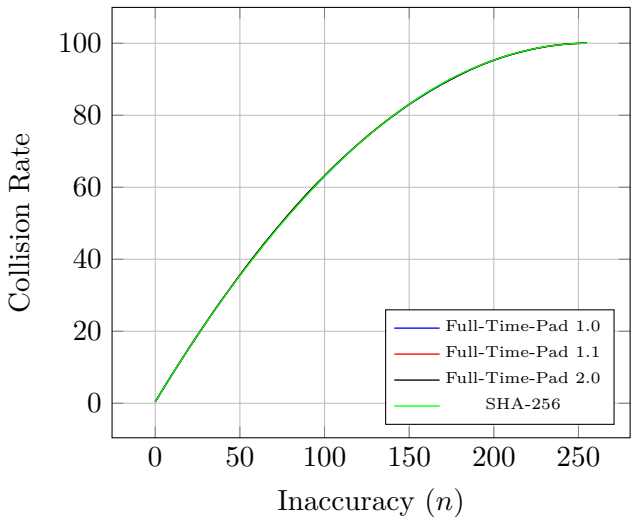
In `test/significant_perm_byte.cpp`, instead of comparing if values are equal, but rather approximately equal with accuracy of $\pm n$. FTP consistently provided similiar results to SHA-256. Also no significant bytes present as proven by changing one particular byte at a time while every other byte remained constant. This means that there isn't a differential statistical pattern.

n	Full-Time-Pad (%)	SHA-256 (%)
0	0.3883	0.3923
5	4.2678	4.224
10	8.0342	8.0322
15	11.7421	11.7403
20	15.4626	15.4287
25	18.8071	18.8382

Table 2: Comparision Accuracy VS. Collision Rate

Results of the collision rates of SHA-256 and FTP are approximately equal given lower accuracy ($\pm n$). This suggests that there aren't any cryptoanalysis exploit. These numbers are average of 255 tries to reduce noise.

Figure 5: Collision Rate (%) vs. Inaccuracy (n)



Since the results on figure 5 of FTP and SHA-256 are equivelant, there are no differential cryptoanalysis exloit vulnerabilities. The smoothness of the trend proves continuity, change in collision rate for n and n+1 must be less than 2%, a single exception every now and then in test trials may be disregarded if not recurring with a pattern. The points for this graph is evaluated by running `test/significant_perm_byte -d -<version number>`. All 4 algorithms have the same result, thus they overlap.

Figure 6: Δ = |SHA-256 - FullTimePad 2.0| V.S. Inaccuracy (n)

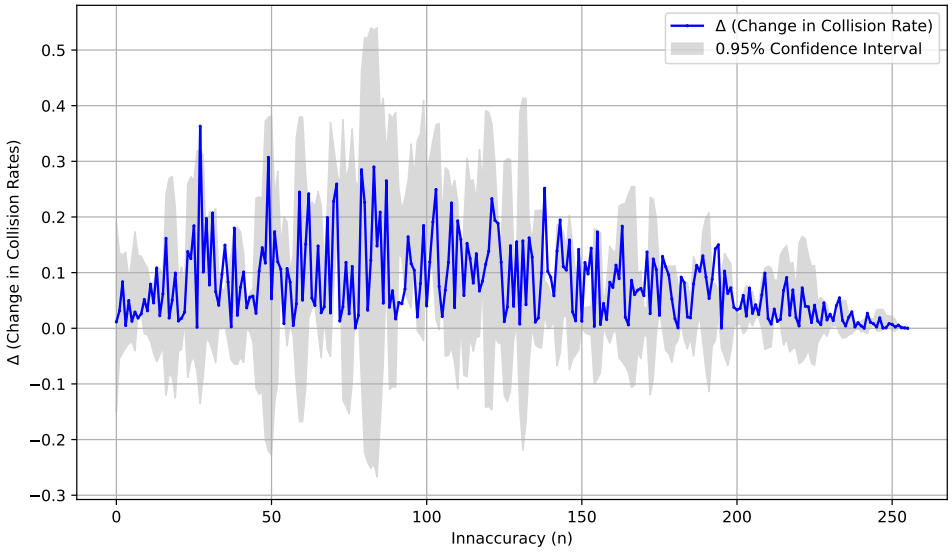


Figure 6 shows the difference (Δ) between SHA-256 hashing algorithm and the `FullTimePad 2.0`. The noise pattern is also similiar for 2 arbitrary sets of secure one-way algorithm statistics which show that graph is entirely noise. Since numbers on the y-axis are very small, Δ is negligible and therefore there isn't a pattern.
SHA-256 is chosen for comparision solely because it is proven to be secure.

2.8 Reverse Engineering the Transformation

Reversing the transformation is demonstrated in `test/reverse.cpp`. The reversed key and the original key are very different. Therefore, reverse engineering the algorithm isn't possible.

2.9 Side-Channel Attacks

If the key is recovered from a side-channel attack, then all plaintexts can be found. However this isn't a pressing issue as all cryptographic algorithms have this flaw. In contrast, without this flaw, a new key would have to be generated per plaintext as demonstrated in OTP which is inefficient.

2.10 Collision-Resistance

Collision resistance data has been shown to be similiar to SHA-256 on section 2.7. The series of tests conducted in `test/significant_perm_byte.cpp` show the continuity and lack of patterns on the rate of collision as 1-bit of input is modified per trial. The tests cover both random and uniform inputs to find patterns. It's important to note that the tests have been verified to be accurate by providing instances where failure is expected. Therefore, successful results from collision tests prove the strength of avalanche effect present in the transformation algorithm of FTP 1.0, 1.1, 2.0.

collision rate, as any byte of the 32-byte key is modified by 1-bit.

$$\texttt{collision_rate} = \frac{\texttt{\#collisions}}{32}$$

where `#collisions` is the number of bytes where `previous key`$_i$ and `modified key`$_i$ are equal

2.10.1 Different Permutation Matrices

Permutation matrices that are either randomly or logically derived do not generate a big difference in collision resistance. However, any type of linear permutation leaves linear trails behind and must be taken care of in other operations present in the transformation algorithm.

2.10.2 Number of Rounds

Number of rounds for FTP 1.0 & 1.1 is determined by the number of rows in the permutation matrix. As using a permutation twice will cancel out. For FTP 2.0, the number of rounds are minimized through testing results.

2.10.3 Constant - r - Dynamic Rotation Constant

Cicadas only surface once every 17 years. Assuming their predator only surfaces once every 2 years, they will only meet each other once every 34 years. Since 17 is a prime number, they will barely ever meet. However, if they surfaced once every 18 years, while their predator once every 2 years. Then every 18 years they would meet. This would also make it easier for the predator to synchronize their emergence timing.
let x be the time interval for when the cicadas surface
let y be the time interval for when the predators surface
We can deduce:
$GCD(x, y) = 1$, then they meet up every $x \times y$ years.
Whereas if $GCD(x, y) \neq 1$, then they meet up every x years (assuming $x > y$).

Bitwise rotation operation should also be done with either a prime number or a number where $GCD(x, y) = 1$ to maximize the collision resistance.

Here is a counterexample as proof
let w be 32-bit number
let $q = 8$ be rotation count
rotation can be represented by: $w = w \lll q$
If this rotation was repeated for 4 iterations, then the rotations would cancel out and rotated w (w') and w would be equal. Make sure that $GCD(q, 32) = 1$.

If there's only one rotation number, the likelihood of finding a pattern increases. Using a single rotation number is not sufficient, but if the bits were shuffled using a more complex method, the process becomes harder to reverse. Even though each method is reversible on its own, combining them makes it significantly more difficult to undo. The use of bitwise rotations on 32-bit segments combined with byte-level permutations creates excellent diffusion.

Rotation numbers that were selected is represented by r:

$$r = \begin{pmatrix} 23 & 5 & 17 & 31 & 13 \end{pmatrix}$$

There are 5 numbers in r, this is because $16 \mod 5 \neq 0$, after 16 iterations in the transformation algorithm, all values of k aren't rotated the same number of times.

if a 32-bit unsigned integer (x) is rotated by r

$$\sum_{i=0}^{4} r_i = 89$$

The total rotation is 89 (prime). $x \ggg 89$ would produce a net rotation of $x \ggg 25$, since $89 \mod 32 = 25$. Knowing that $GCD(25, 32) = 1$, the minimum number of times where the expression $x \ggg 89$ is repeated until $x \ggg 89 = x$ is 32 times since $GCD(25 \times 32, 32) = 32$. Considering that there are other operations than bit shifts, the total bit rotation (89) reflects the transformation of bits better than the net rotation (25).

$\therefore$ r must contain multiple numbers.
FTP is designed so that $|r_i - r_{i-1}|$ results in a different magnitude for all elements of r so that diffusion from bitwise rotations are maximized.
Derivation of r is as follows: make a set (r) of all prime numbers from 0-32, remove elements based on similarity of results of $|r_i - r_{i-1}|$ (e.g. $23 - 5 = 18$, $|5 - 17| = 12$). A good net rotation would be prime. FTP consists of both bitwise right rotate ($\ggg$) and bitwise left rotate ($\lll$), in order to properly mix the results.

3 Hashing

The 'hashing' algorithm is used for manipulating the key per plaintext with proper confusion and avalanche effect. The main reason why the key can be reused is due to the algorithm's strong avalanche effect as well as the lack of statistical patterns.
To maximize the effects of confusion in FTP, half of the key is modified per iteration. Every byte of the key is transformed a total of 8 times ($\frac{16\texttt{iterations}}{8 \div 4}$). FTP 1.0 & 1.1 consists of 16 iterations and since $16 \mod 2 = 0$, all bytes are transformed the same number of times which would lower the possibility of a statistical exploit that could target less transformed bytes of the key.
Half of the key should be modified per iteration because classical cryptographic algorithms (non-quantum) rely on computational complexity for their security. So more transformations the better. Although the minimum work to get a secure transformed key is enough,
it's often preferred to have extra insurance rounds for further security at the expense of computation speed.

3.1 Dynamic Matrix Permutation

Considering the use of `reinterpret_cast` in C++ and unions in C for representing the bytearray (key) as 32-bit segments (for confusion operations), the endianness of the device will require a different matrix. the matrix given in this documentation is the big endian version as FTP is designed to be big-endian. The little endian version requires a new permutation matrix where the byte order every 32-bits is reversed. Both matrices can be found in the C++ implementation of `Full-Time-Pad` and the python derivation program `test/perm.py`.
This generates a powerful permutation matrix.

3.1.1 Derivation

Algorithm 1 Dynamic Permutation Matrix Derivation Pseudo-code

Input: an array of incrementing numbers (0-31) A
Output: Most Efficient Permutation Matrix V (16×32)
Begin
$P \leftarrow$ `copy of A`
for $k = 0$ to 4 **do**
    for $i = 0$ to 8 **do**
        $P_i \leftarrow A_{i\times4}$
        $P_{i+8} \leftarrow A_{i\times4+1}$
        $P_{i+16} \leftarrow A_{i\times4+2}$
        $P_{i+24} \leftarrow A_{i\times4+3}$
    end for
    $A \leftarrow$ `copy of P`
    $V.append(P)$
    $C \leftarrow$ `copy of P`
    for $m = 0$ to 3 **do**
        for $i = 0$ to 8 **do**
            for $n = 0$ to 4 **do**
                $P_{i\times4+n} \leftarrow C_{(1+n+m) \mod 4 + i\times4}$
            end for
        end for
        $V.append(P)$
    end for
    $A \leftarrow$ `copy of P`
end for
Return V

3.1.2 Dynamic Permutation Matrix Values

$$\left\{\begin{array}{cccccccccccccccccccccccccccccccc}
0 & 4 & 8 & 12 & 16 & 20 & 24 & 28 & 1 & 5 & 9 & 13 & 17 & 21 & 25 & 29 & 2 & 6 & 10 & 14 & 18 & 22 & 26 & 30 & 3 & 7 & 11 & 15 & 19 & 23 & 27 & 31 \\
4 & 8 & 12 & 0 & 20 & 24 & 28 & 16 & 5 & 9 & 13 & 1 & 21 & 25 & 29 & 17 & 6 & 10 & 14 & 2 & 22 & 26 & 30 & 18 & 7 & 11 & 15 & 3 & 23 & 27 & 31 & 19 \\
8 & 12 & 0 & 4 & 24 & 28 & 16 & 20 & 9 & 13 & 1 & 5 & 25 & 29 & 17 & 21 & 10 & 14 & 2 & 6 & 26 & 30 & 18 & 22 & 11 & 15 & 3 & 7 & 27 & 31 & 19 & 23 \\
12 & 0 & 4 & 8 & 28 & 16 & 20 & 24 & 13 & 1 & 5 & 9 & 29 & 17 & 21 & 25 & 14 & 2 & 6 & 10 & 30 & 18 & 22 & 26 & 15 & 3 & 7 & 11 & 31 & 19 & 23 & 27 \\
12 & 28 & 13 & 29 & 14 & 30 & 15 & 31 & 0 & 16 & 1 & 17 & 2 & 18 & 3 & 19 & 4 & 20 & 5 & 21 & 6 & 22 & 7 & 23 & 8 & 24 & 9 & 25 & 10 & 26 & 11 & 27 \\
28 & 13 & 29 & 12 & 30 & 15 & 31 & 14 & 16 & 1 & 17 & 0 & 18 & 3 & 19 & 2 & 20 & 5 & 21 & 4 & 22 & 7 & 23 & 6 & 24 & 9 & 25 & 8 & 26 & 11 & 27 & 10 \\
13 & 29 & 12 & 28 & 15 & 31 & 14 & 30 & 1 & 17 & 0 & 16 & 3 & 19 & 2 & 18 & 5 & 21 & 4 & 20 & 7 & 23 & 6 & 22 & 9 & 25 & 8 & 24 & 11 & 27 & 10 & 26 \\
29 & 12 & 28 & 13 & 31 & 14 & 30 & 15 & 17 & 0 & 16 & 1 & 19 & 2 & 18 & 3 & 21 & 4 & 20 & 5 & 23 & 6 & 22 & 7 & 25 & 8 & 24 & 9 & 27 & 10 & 26 & 11 \\
29 & 31 & 17 & 19 & 21 & 23 & 25 & 27 & 12 & 14 & 0 & 2 & 4 & 6 & 8 & 10 & 28 & 30 & 16 & 18 & 20 & 22 & 24 & 26 & 13 & 15 & 1 & 3 & 5 & 7 & 9 & 11 \\
31 & 17 & 19 & 29 & 23 & 25 & 27 & 21 & 14 & 0 & 2 & 12 & 6 & 8 & 10 & 4 & 30 & 16 & 18 & 28 & 22 & 24 & 26 & 20 & 15 & 1 & 3 & 13 & 7 & 9 & 11 & 5 \\
17 & 19 & 29 & 31 & 25 & 27 & 21 & 23 & 0 & 2 & 12 & 14 & 8 & 10 & 4 & 6 & 16 & 18 & 28 & 30 & 24 & 26 & 20 & 22 & 1 & 3 & 13 & 15 & 9 & 11 & 5 & 7 \\
19 & 29 & 31 & 17 & 27 & 21 & 23 & 25 & 2 & 12 & 14 & 0 & 10 & 4 & 6 & 8 & 18 & 28 & 30 & 16 & 26 & 20 & 22 & 24 & 3 & 13 & 15 & 1 & 11 & 5 & 7 & 9 \\
19 & 27 & 2 & 10 & 18 & 26 & 3 & 11 & 29 & 21 & 12 & 4 & 28 & 20 & 13 & 5 & 31 & 23 & 14 & 6 & 30 & 22 & 15 & 7 & 17 & 25 & 0 & 8 & 16 & 24 & 1 & 9 \\
27 & 2 & 10 & 19 & 26 & 3 & 11 & 18 & 21 & 12 & 4 & 29 & 20 & 13 & 5 & 28 & 23 & 14 & 6 & 31 & 22 & 15 & 7 & 30 & 25 & 0 & 8 & 17 & 24 & 1 & 9 & 16 \\
2 & 10 & 19 & 27 & 3 & 11 & 18 & 26 & 12 & 4 & 29 & 21 & 13 & 5 & 28 & 20 & 14 & 6 & 31 & 23 & 15 & 7 & 30 & 22 & 0 & 8 & 17 & 25 & 1 & 9 & 16 & 24 \\
10 & 19 & 27 & 2 & 11 & 18 & 26 & 3 & 4 & 29 & 21 & 12 & 5 & 28 & 20 & 13 & 6 & 31 & 23 & 14 & 7 & 30 & 22 & 15 & 8 & 17 & 25 & 0 & 9 & 16 & 24 & 1
\end{array}\right\}$$

3.2 Confusion - ARX

For the confusion aspect of the key transformation algorithm, ARX operations are used.

3.2.1 A - Modular Addition

For the modular addition, a constant array of 8 values is used (`A`):

$$\texttt{encryption index} \gg 32$$
$$\texttt{encryption index \& 0xffffffff}$$
$$\texttt{0x119f904f}$$
$$\texttt{0x73d44db5}$$
$$\texttt{0x3918fa83}$$
$$\texttt{0x5546b403}$$
$$\texttt{0x216c46df}$$
$$\texttt{0x64997dfd}$$

`A` is derived as follows:
Start with the message: "**cipher**" (6 digit message with 6 unique characters)
Take every ascii encoding of each character of the message and calculate to the 32nd power and take the first 32-bits:
Example:
"c": 99 in ascii encoding
the first 32-bits of 99^{32} = `0x119f904f`
The word **cipher** and $\mathbf{A}_i$ is arbitrary. In other words, they're "nothing up my sleeve numbers". This is an initial array. In the transformation algorithm, every $\mathbf{A}_i$ is modified twice (in 16 iterations). This provides some confusion. The reason why the first 2 values are the 64-bit encryption index is because this was the most efficient way to diffuse in the encryption index.

3.2.2 R - Bitwise Rotation

"Using the values determined for r in Section 2.10.3, rotate each value one at a time for diffusing bits non-uniformly and linearly. FTP 1.0 consists of 2 bitwise right rotates (BRR) & 1 bitwise left rotate (BLR) offering a total rotation count of 48. FTP 1.1 consists of 1 BRR and 1 BLR with a total rotation count of 32. FTP 2.0 consists of 1 BRR and 1 BLR wit ha total rotation count of 20. Using both BLR and BRR isn't counter productive since it's not applied simultaneously. It is potentially redundant to use both but the idea was to use more operations to add more complexity.

3.2.3 X - XOR

XOR operator offers linear manipulation for merging values. Using alongside addition since adding both linear and non-linear operations offers stronger confusion.

4 Performance

In terms of performance, FTP 1.0 is quite slow compared to ChaCha20 and AES. The main reason for the poor performance is the 16 rounds of permutation. Not all versions of FTPvalue speed, FTP 1.0 purely focuses on security. When dealing with high security dependent objectives such as password sharing, using slower algorithms with more insurance rounds is preferred since it's safer from severe & powerful attacks.
Benchmark tests show the following results:

```
-----TESTING SPEED-----
TESTING TRANSFORMATION VERSION 1.0:
Computation Time for 1000000 key transformations:  0.641983 seconds
Hashrate:  1557674 keys per second
```

```
TESTING TRANSFORMATION VERSION 1.1:
Computation Time for 1000000 key transformations:  0.526699 seconds
Hashrate:  1898618 keys per second

TESTING TRANSFORMATION VERSION 2.0:
Computation Time for 1000000 key transformations:  0.059143 seconds
Hashrate:  19178011 keys per second
```

On the same device (12th Gen Intel(R) Core(TM) i7-12700H, DDR5 RAM). The ChaCha20 benchmark tests yield the following results:

```
Time taken to encrypt 1000000 32-byte messages:  0.160442 seconds
Encryption rate:  6232781 keys per second
```

FTP 2.0 is 3 times faster than ChaCha20. However, ChaCha20 has 64-byte keystreams while FTP has 32-bytes. FTP 2.0 is 3x faster for encrypting unique plaintexts under 32 bytes (For long plaintexts, 1.5x). These tests are under the assumption that optimizations are equivalently effective and heavily used. Although benchmark tests with Salsa12 and FTP 2.0 are similiar for a 64-byte output, this algorithm could be updated to provide a 64-byte output without adding 2 times the computation time So comparing benchmark tests relative to the ratio of output size is misleading. More so because it is designed to be a 32-byte output. Accurate comparison requires more work.
Since implementation of FTP 1.0 & 1.1 aren't optimized fully, the benchmark tests are misleading.
On future devices, SIMD instructions could be expected to improve and provide different benchmarks disproportionally so comparison is inaccurate.

5 Cipher

5.1 Transformation - Version 1.0 - Prioritize Complexity

Algorithm 2 The key transformation operation per encryption

Input: 32-byte bytearray key (`key`), `64-bit encryption_index`
Output: Transformed 32-byte bytearray key (k)
Begin
// `keysize` = 32, length of input key is 32-bytes
k ← endian_8_to_32_arr(`key`)
for $i = 0$ to 16 **do**
    index ← $i \ll 2$
    i1mod ← index $\mod 8$
    i2mod ← index $+ 1) \mod 8$
    i3mod ← index $+ 2) \mod 8$
    i4mod ← index $+ 3) \mod 8$
    imod8 ← $i \mod 8$
    imod9 ← $(i + 1) \mod 8$
    rmod ← $i \mod 5$

    $k_{\text{i1mod}} \leftarrow k_{\text{i1mod}} + A_{\text{imod8}} + k_{\text{i1mod}} \ggg r_{\text{rmod}}$
    $\text{sum} \leftarrow \sum_{j=0}^{8} k_j$
    $A_{\text{imod9}} \leftarrow A_{imod9} \oplus \text{sum}$
    $k_{\text{i2mod}} \leftarrow k_{i2mod} + A_{imod9} + k_{i2mod} \lll r_{rmod}$
    $A_{\text{imod8}} \leftarrow A_{\text{imod8}} \oplus k_{\text{i2mod}} + k_{\text{i1mod}} \ggg r_{(i+1) \mod 5}$
    $k_{\text{i3mod}} \leftarrow A_{\text{imod8}} \oplus k_{\text{i3mod}} + A_{\text{imod9}} \oplus k_{\text{i4mod}}$
    $k_{\text{i4mod}} \leftarrow A_{\text{imod8}} \oplus k_{\text{i4mod}} + A_{\text{imod9}} \oplus k_{\text{i3mod}}$

    // Permute the bytearray key
    `key` ← dynamic_permutation(`key`, i);
    increment `encryption_index` by 1 for each plaintext
end for
Return `key`

5.2 Transformation - Version 1.1 - Prioritize Speed & Complexity

This version of the transformation offers more speed less complexity. This is preferred to the previous option since version 1.0 contains potentially redundant/extra operations, FTP 1.0 is better suited for when speed doesn't matter as much as security, or when slower algorithms are appreciated.

Algorithm 3 The key transformation operation per encryption

Input: 32-byte bytearray key (`key`), `64-bit encryption_index`
Output: Transformed 32-byte bytearray key (k)
Begin
// `keysize` = 32, length of input key is 32-bytes
k ← endian_8_to_32_arr(`key`)
for $i = 0$ to 16 **do**
    index ← $i \ll 2$
    i1mod ← index mod 8
    i2mod ← index + 1) mod 8
    i3mod ← index + 2) mod 8
    i4mod ← index + 3) mod 8
    imod8 ← i mod 8
    imod9 ← $(i + 1)$ mod 8
    rmod ← i mod 5

    $k_{\text{i1mod}} \leftarrow k_{\text{i1mod}} + A_{\text{imod8}} + k_{\text{i1mod}} \ggg r_{\text{rmod}}$
    $\text{sum} \leftarrow \sum_{j=0}^{8} k_j$
    $A_{\text{imod9}} \leftarrow A_{\text{imod9}} \oplus \text{sum}$
    $k_{\text{i2mod}} \leftarrow k_{\text{i2mod}} + A_{\text{imod9}} + k_{\text{i2mod}} \lll r_{\text{rmod}}$
    $A_{\text{imod8}} \leftarrow A_{\text{imod8}} \oplus k_{\text{i2mod}}$
    $k_{\text{i3mod}} \leftarrow A_{\text{imod8}} \oplus k_{\text{i3mod}}$
    $k_{\text{i4mod}} \leftarrow A_{\text{imod8}} \oplus k_{\text{i4mod}}$

    // Permute the bytearray key
    `key` ← dynamic_permutation(`key`, i);
end for
increment `encryption_index` by 1 for each plaintext
Return `key`

5.3 Transformation - Version 2.0 - Prioritize Speed

This version of the transformation prioritizes speed. FTP 2.0 is preferred to FTP 1.1 for cases where complexity isn't the primary focus. This is still considered secure, but lacks the extra complexity found in FTP 1.0. FTP is designed to offer less rounds, and less bytes modified per round, but heavier manipulation per round.

Algorithm 4 The key transformation operation per encryption

Input: 32-byte bytearray key (`key`), `64-bit encryption_index`
Output: Transformed 32-byte bytearray key (k)
Begin
// `keysize` = 32, length of input key is 32-bytes
k ← endian_8_to_32_arr(`key`)
for $i = 0$ to 10 **do**
    index ← $i \ll 2$
    i1mod ← index mod 8
    i2mod ← index + 1) mod 8
    i3mod ← index + 2) mod 8
    i4mod ← index + 3) mod 8
    imod8 ← i mod 8
    imod9 ← $(i + 1)$ mod 8
    rmod ← i mod 5

    $k_{\text{i1mod}} \leftarrow k_{\text{i1mod}} + A_{\text{imod8}} + k_{\text{i1mod}} \ggg r_{\text{rmod}}$
    sum ← $\sum_{j=0}^{8} k_j$
    $A_{\text{imod9}} \leftarrow A_{\text{imod9}} \oplus \text{sum}$
    $k_{\text{i2mod}} \leftarrow k_{\text{i2mod}} + A_{\text{imod9}} + k_{\text{i2mod}} \lll r_{\text{rmod}}$
    $A_{\text{imod8}} \leftarrow A_{\text{imod8}} \oplus k_{\text{i2mod}}$
    $k_{\text{i3mod}} \leftarrow A_{\text{imod8}} \oplus k_{\text{i3mod}}$
    $k_{\text{i4mod}} \leftarrow A_{\text{imod8}} \oplus k_{\text{i4mod}}$

    // Permute the bytearray key only 2 times, don't permute before last iteration
    if i mod 4 = 0 and $i \neq 8$ **then**
        `key` ← dynamic_permutation(`key`, i);
    end if
end for
increment `encryption_index` by 1 for each plaintext
Return `key`

5.4 dynamic permutation

Algorithm 5 The dynamic_permutation() in the transformation function

Input: 32-byte bytearray (B), permutation matrix (V), iteration index (i)
Output: 32-byte permuted bytearray (B)
let P be placeholder vector declared with length of 32-bytes
for $j = 0$ to 32 **do**
    $P_j = B_{V_{i_j}}$
end for
Return P

dynamic permutation function permutes the bytearray key using the permutation matrix found in section 3.1.2.

5.5 endian_8_to_32_arr

Algorithm 6 The endian_8_to_32_arr() in the transformation function

Input: 32-byte bytearray (B)
Output: 32-byte bytearray (B) permuted to flip endianness if necessary
if big endian system **then**
    for $j = 0$ to 32 **do**
        swap B_i with B_{i+3}
        swap B_{i+1} with B_{i+2}
    end for
end if
Return reinterpret_cast <uint32_t* >(B)

This endianness conversion is designed to convert a little-endian reinterpret casted pointer into the big-endian version so that both little-endian and big-endian systems are compatible.

5.6 Key Reuse

A 64-bit nonce labeled an encryption index is used for key reusability. The difference between a nonce and encryption index is that an encryption index cannot be random. if a random nonce was used instead, this could lead to collisions since root k for $g(2^{64}, k) = 5,056,937,540$ where g is the solution to the birthday problem defined in section 2.2.1. This means that there is a 50% chance of collision when 5 billion 64-bit nonces are used. 161,822,001,280 bytes of data would need to be encrypted for a successful brute-force to be 50% likely. In practice, using random nonces would increase chances of collisions significantly. Safest practice is to increment the encryption index.
The first 2 values of A is the encryption index. Encryption index must be located in the first half of A to provide more iterations to diffuse in the encryption index.

5.6.1 Multi-Threading

When multiple threads are encrypting using the same key, the encryption index needs to be an atomic variable. If using an atomic number isn't possible (e.g. threads cannot communicate efficiently), then use one the following methods:

Algorithm 7 Unique nonces for Multi-threading without thread communication

Input: thread count (t), thread ID (n)
if first `encryption_index` **then**
    `encryption_index` $= n$
else
    Add t to `encryption_index`
end if
Output: Nonce (`encryption_index`)

Algorithm 8 Unique nonces for Multi-threading without thread communication

Input: thread count (t), thread ID (n), last encryption index used on this thread (m), $m > 0$, ordered set of t prime numbers (P)
let `encryption_index` $\leftarrow n + m + 1$
let $p \in P, p \neq P_n$
while $\gcd(\texttt{encryption_index}, p) > 1$ **do**
    `encryption_index`$+= P_n$
end while
Output: Nonce (`encryption_index`)

These algorithm are only valid if number of threads used is not growing. Otherwise, every time a new thread is created, the algorithm needs to be recalled with a new t. If thread count is not constant, set thread count to maximum possible number before executing the algorithm.

Algorithm-8 is inefficient and slow so it should be avoided in practice. $\therefore$ Use **Algorithm 7** for generating incremental nonces without thread communication.

5.7 Long Plaintexts

When encrypting long plaintexts, using encryption modes such as Galois Counter Mode (GCM) is always the best option but it is also secure to increment the encryption index per 32-byte plaintext block.

5.8 Encryption/Decryption operation

Encryption operation is the same as decryption

Algorithm 9 Encrypt/Decrypt transform Function

Input: plaintext (p), length (ℓ), 32-byte original key (k), encryption index/nonce (`encryption_index`)
Output: ciphertext (c)
for each 32-byte segment of plaintext **do**
    // create a copy of unmodified k, and transform it using an incrementing unique `encryption_index`
    // Then, encrypt using plaintext xor transformed key = ciphertext
    copy = copy(k)
    $c \leftarrow c \,||\, p \oplus$ transformation(copy, `encryption_index`)
end for

// For the remainder:
$c \leftarrow c \parallel (p \oplus \text{transformation}(\text{copy}, \texttt{encryption_index}))$
Return c

6 Conclusion

The algorithm proposed in this study that builds on cryptographic principles based on OTP, using byte-permutations and ARX operations is both introduced and tested for randomness, brute force, Joux's attack, related key attack, time-based attack, linear cryptoanalysis, differential cryptoanalysis and reverse engineering vulnerabilities. Although all tests successfully passed. This algorithm requires further tests to verify the

security of FTP. The benchmark tests need to be updated using improved implementation of FTP 1.0 & 1.1. FTP 2.0 offers competitive benchmarking tests that compete with industry standard algorithms.

A Pre-requisite Terminology

Key	32-byte random array that's transformed, then hashed before XORed with the plaintext to encrypt
Symmetric	Same key is used for encryption and decryption
Stream	Plaintext is encrypted without separating it into blocks
Plaintext	Plain data before encryption
Ciphertext	Encrypted plaintext
Cipher	Encryption algorithm. Plaintext is transformed into a ciphertext that can only be reversed with a key
Diffusion	plaintext/key is spread out in the ciphertext
Confusion	The ciphertext has no possible statistical analysis, or cryptoanalysis to determine the plaintext
Bit	0 or 1. Smallest discrete unit for computation
Byte	8-bit number
Avalanche Effect	An aspect of diffusion. If smallest unit (1 bit) of data is changed, the ciphertext changes in an unrecognizable way.
Permutation	transposing (moving) elements of a vector without actually changing their values. A reversible operation

References

[1] Claude E. Shannon. *A Mathematical Theory of Cryptography*. Bell System Technical Journal, 1945. URL: https://www.bibsonomy.org/bibtex/2f4083c0272b2f79fe077da0227e43b5c/overleaf.

[2] Michael Sung. *SIMD Parallel Processing*. Feb. 2000. URL: http://www.ai.mit.edu/projects/aries/papers/writeups/darkman-writeup.pdf.

[3] J. Palms. *Efficient Permutation Instructions for Fast Software Cryptography*. Dec. 2001. URL: http://palms.ee.princeton.edu/PALMSopen/lee01efficient.pdf.

[4] Antoine Joux. *Multicollisions in iterated hash functions. Application to cascaded constructions*. Mar. 2004. URL: https://www.iacr.org/archive/crypto2004/31520306/multicollisions.pdf.

[5] Wikipedia contributors. *Birthday Problem*. May 2025. URL: https://en.wikipedia.org/w/index.php?title=Birthday_problem&oldid=1291691534.

[6] Wikipedia contributors. *Single Instruction, Multiple Data*. Mar. 2025. URL: https://en.wikipedia.org/wiki/Single_instruction,_multiple_data.